DATA 612 · Statistical Programming in R · Week 14

14.2 Iteration: Loops and purrr

Pre-allocate a for-loop. map() walks a list. across() often replaces a column loop.

1. A for-loop, pre-allocated

```
for (i in seq_along(x)) {
  # use x[[i]]
}
```

`seq_along(x)` is `1, 2, …, length(x)`. Prefer it to `1:length(x)`, which misbehaves on an empty `x`. Growing a vector with `c()` inside the loop is slow. Make the output first, then fill it.

```
x <- 1:5
cumvec <- vector("double", length(x))
total <- 0
for (i in seq_along(x)) {
  total <- total + x[[i]]
  cumvec[[i]] <- total
}
```

`vector("double", n)` is `n` zeros. You overwrite them.

---

2. `while` when `n` is unknown

A `for` loop needs the sequence up front. A `while` loop keeps going until a condition is false:

```
nheads <- 0
flips <- 0
while (nheads < 3) {
  flips <- flips + 1
  if (rbinom(1, size = 1, prob = 0.5) == 1) {
    nheads <- nheads + 1
  }
}
```

Use `while` when you cannot say how many steps you need.

---

3. `across()` often replaces a column loop

Looping over `mtcars` columns to take a mean works. `across()` is the dplyr version, and it stays in a pipeline:

```
means <- vector("double", ncol(mtcars))
for (i in seq_along(mtcars)) {
  means[[i]] <- mean(mtcars[[i]], na.rm = TRUE)
}

mtcars |>
  summarize(across(where(is.numeric), ~ mean(.x, na.rm = TRUE)))
```

Reach for a loop when the result is not one number per column: a model, a plot, a file.

---

4. `purrr::map`

`map()` applies a function to each element and returns a list. Typed variants return an atomic vector: `map_dbl()`, `map_chr()`, `map_lgl()`.

`.f` is a function name, **not** a call. `map_dbl(mtcars, mean)` works. `map_dbl(mtcars, mean())` does not.

```
map_dbl(mtcars, mean)
map_dbl(mtcars, mean, na.rm = TRUE)
```

A formula `~` is a short anonymous function. `.x` is the current element:

```
map_dbl(mtcars, ~ mean(.x, na.rm = TRUE))
```

---

5. `map2` and `pmap`

`map2(.x, .y, .f)` walks two vectors together. `pmap()` walks a list of arguments:

```
mu <- c(-10, 0, 10)
sigma <- c(1, 1, 2)
map2(mu, sigma, rnorm, n = 5)

args <- list(n = 5, mean = mu, sd = sigma)
pmap(args, rnorm)
```

---

6. `keep` / `discard` and split-plus-map

`keep()` holds the elements where a test is true. `discard()` drops them.

```
iris |>
  keep(is.numeric) |>
  map_dbl(mean)
```

`split()` turns a data frame into a list of data frames, one per group. Then `map()` fits a model on each:

```
mtcars |>
  split(mtcars$cyl) |>
  map(~ lm(mpg ~ wt, data = .x)) |>
  map(summary) |>
  map_dbl("r.squared")
```

A string (or an integer) as `.f` extracts that name (or position) from each list element.

---

7. Practice

1. The first two Fibonacci numbers are 0 and 1. Each later number is the sum of the previous two. Use a loop to compute the first 100. Sanity check: `log2()` of the 100th number is about 67.57.
2. `map_dbl()` means of every column of `mtcars`.
3. Number of unique values in each column of `iris`. Repeat with `across()`. `typeof()` of each column of `nycflights13::flights` with `map_chr()`, then with `across()`.
4. p-value of `t.test(mpg ~ am)` within each `cyl` group of `mtcars`.
5. In three lines, keep the `mtcars` columns whose mean is greater than 10 and report those means (`keep()` then `map_dbl()`). You should get four variables.
6. Standard deviation of each numeric column of `iris` with a for-loop, then with `across()`.